1. Delivery strategy

The 24-week plan builds a demonstrable, safe academic platform in dependency order. Each phase ends with an exit gate: a reviewable artifact, automated checks, and a decision to proceed. The project retains a simulation boundary: no real emergency service is integrated or contacted.

- Team roles: product/architecture lead, backend/data lead, frontend lead, ML/experimentation lead, QA/DevOps lead. One student may hold multiple roles, but each deliverable has a named accountable owner.

- Every sprint ends with a working increment, a test run, an updated risk register, and a documented architecture decision where a major choice changes.

2. Operating model and definition of done

Work uses short-lived feature branches and protected main. Each change is linked to an issue, reviewed by a peer, tested in CI, documented where user-visible, and deployed to staging before it is declared complete.

- Definition of done: acceptance criteria met; unit/integration tests pass; lint/security checks pass; no secrets or production data committed; observability added; API/event migration documented; rollback known; reviewer approval recorded.

- Code ownership: backend owns schemas/API/event contracts; ML owns data/model cards; frontend owns PWA accessibility; DevOps owns pipelines/IaC; QA owns acceptance evidence.

- Issue templates: feature, defect, security finding, experiment, dataset intake, architecture decision, and operational incident.

Phase	Weeks	Goal	Exit gate
1. Foundations	1-4	Reproducible repo, local stack, standards	Fresh clone runs Compose and CI
2. Core platform	5-8	Secure incident workflow and durable data	Audited incident lifecycle passes integration suite

3. User experience	9-12	Operator and verifier PWAs	End-to-end task/evidence/decision demo
4. ML intelligence	13-16	Perception, fusion and comparison	Reproducible evaluation and model cards
5. TMVN and explainability	17-20	Tasking, escalation, operator control	Simulation demonstrates ambiguous-case loop
6. Staging and review	21-24	Hardened AWS pilot and final evidence	Load/security/restore/review demo complete

3. Phase 1- Foundations (weeks 1-4)

Establish a monorepo with frontend, backend, ml-services, infra, docs and test directories. Compose runs PostgreSQL/PostGIS, Kafka, Neo4j, MinIO/S3-compatible storage, gateway and observability stubs. Create environment templates with secret names only, never values.

- Week 1: backlog, architecture decision records, repository, contribution guide, branch protections.
- Week 2: Compose, health checks, base Spring Boot/React/Python services, lint/format/test commands.
- Week 3: Flyway migrations, mock-source generator, OpenAPI skeleton, Kafka topic convention, correlation IDs.
- Week 4: GitHub Actions, SAST/dependency/container scans, staging baseline and walkthrough.
- Acceptance: a new contributor can run the stack, create mock events, inspect logs and pass CI without manual database setup.

4. Phase 2- Core platform (weeks 5-8)

Implement the system of record before advanced ML. The Incident Service owns lifecycle transitions; the Evidence Service owns upload policy; the Decision Service owns approval/override; the Audit Service appends immutable events. The gateway enforces OAuth2/JWT and RBAC.

- Week 5: PostGIS incident/observation/confidence schema and indexes; Neo4j relationship model.
- Week 6: Kafka producers/consumers, outbox, idempotency, retry and dead-letter processing.
- Week 7: OAuth2/JWT, role matrix, audit events, upload presigning and validation.
- Week 8: REST and event contract tests, API documentation, duplicate/replay and permission acceptance tests.
- Acceptance: incident creation through approved decision can be replayed, traced, and denied correctly by role.

5. Phase 3- User experience (weeks 9-12)

Build thin vertical slices first: the operator map reads mock incidents, opens an explanation, and records a decision; then the verifier PWA receives a task and submits evidence. Avoid building administration breadth before this core loop works.

- Week 9: design system, sign-in, role routing, accessible map/list view.
- Week 10: incident detail, evidence timeline, confidence explanation, approval/adjust/override flow.
- Week 11: verifier task flow, permission/consent screen, location/evidence upload, offline draft.
- Week 12: notifications, administration, responsive/accessibility tests, end-to-end Playwright suite.
- Acceptance: demonstration completes operator review and verifier evidence flow on desktop and mobile viewport with keyboard support.

6. Phase 4- ML intelligence (weeks 13-16)

ML work is reproducible and separated from online serving. Dataset intake records license and version. Event/location-based splits are created before feature engineering. MLflow or equivalent records parameters, metrics, artifacts, calibration plots and model cards.

- Week 13: dataset manifests, preprocessing, label checks, leakage tests, baseline data pipeline.
- Week 14: train YOLOv8-S/RT-DETR fire comparison and U-Net/DeepLabV3+ flood comparison.
- Week 15: train DistilBERT/logistic NLP comparison; feature snapshots and calibrated XGBoost fusion baseline.
- Week 16: PyMC Bayesian fusion, credible intervals, threshold policy, drift metrics, versioned inference API.
- Acceptance: repeatable report compares models on held-out event groups; no model is deployed without calibration, model card and rollback artifact.

7. Phase 5- TMVN, explainability and simulation (weeks 17-20)

The Trusted Mobile Verification Network is built as a simulated network with configurable verifier positions, reliability, availability, cost, travel time and response behaviour. The selection score is visible to operators and can be sensitivity-tested.

- Week 17: OSMnx/NetworkX routing, SimPy scenario runner, verifier score and escalation rules.
- Week 18: task creation, notification, expiry, re-assignment, evidence reliability and conflict handling.
- Week 19: evidence fusion, signal contribution display, recommendation generator and audit-linked explanation.
- Week 20: operator approval controls, simulated police/fire/ambulance/shelter dispatch planner, scenario tests.

- Acceptance: low-risk, high-confidence, ambiguous, no-response, conflicting-evidence and override scenarios all produce correct states and audit records.

8. Phase 6- AWS staging, assurance and review (weeks 21-24)

Infrastructure is deployed first to staging using least privilege and separate environment configuration. Production is only a documented target; final academic demonstration remains staging/simulated unless governance explicitly authorises a different environment.

- Week 21: AWS VPC, ECS/Fargate, RDS, S3, Kafka, Secrets Manager, CloudWatch and IaC review.

- Week 22: dashboards, SLO alerts, rate limiting, error tracking, backup and restore drill.

- Week 23: security review, dependency remediation, load/chaos testing, cost review and accessibility testing.

- Week 24: pilot scenarios, final metrics, review demo, user guide, architecture pack and retrospective.

- Acceptance: staged release passes smoke/security/load/restore checks; dashboard shows health; rollback is rehearsed; review evidence is packaged.

9. Data governance, quality and evaluation

Dataset intake is blocked until license, attribution, purpose, personal-data risk, geographic applicability and storage location are recorded. Training data is versioned with checksums; samples with unclear rights or unsafe personal content are excluded. Evidence is never used for model retraining without explicit governance approval.

- Quality targets: API contract coverage 100% for published endpoints; critical workflow end-to-end coverage; no unresolved critical/high security findings at release gate.

- ML report: mAP/precision/recall/F1 for fire; IoU/Dice for flood; F1 for NLP; calibration/Brier/ECE, false dispatch rate, verification latency and cost for fusion.

- Human review: operator overrides and post-scenario adjudication form the gold evaluation set; disagreement is retained as uncertainty, not silently relabelled.

10. Deployment, rollback and operational gates

CI creates immutable tagged images. CD deploys to staging, runs migrations using backward-compatible expand/contract changes, performs smoke and contract checks, then requires manual approval before any protected environment. Feature flags protect new fusion policies and UI actions.

- Rollback: reverse traffic to last healthy image; disable a model through registry alias; pause a faulty consumer; preserve failed events in DLQ; use tested RDS point-in-time recovery only under incident command.

- Release gates: passing CI, peer review, security scan, migration plan, observability, automated tests, staging smoke test, rollback confirmation, and accountable approver.

- Monitoring gates: P95 latency, error rate, Kafka lag, source freshness, dead-letter count, upload failure, task expiry, override rate, model confidence distribution and calibration drift.

11. Risks and mitigations

The plan manages technical and academic risks early rather than leaving them to final integration.

Risk	Mitigation	Owner
Dataset license/availability changes	Intake checklist, mirrors only when permitted, maintain secondary candidates	ML lead
Model overconfidence or leakage	Event-grouped splits, calibration, operator approval, model cards	ML lead
Complex integration delays	Vertical slice first, mocks/adapters, contract tests	Architecture lead

Evidence privacy/security	Short-lived URLs, RBAC, encryption, upload scanning, audit	Backend/DevOps
Cloud cost growth	Budgets, autoscaling caps, staging schedules, cost dashboard	DevOps
Verifier non-response	Simulation, timeout/escalation, fallback operator review	Product lead

12. Requirements traceability and review milestones

The traceability matrix is maintained in the repository and updated every sprint. It links each objective to component, test evidence, and review demonstration. Review 1 demonstrates foundations plus early perception dataset/model work; Review 2 demonstrates the full human verification loop; final review demonstrates evaluation, security, resilience and a controlled pilot.

Requirement	Component	Verification evidence	Review
Multi-source confidence	Adapters + Fusion Service	Event replay + calibration report	R2/Final
Cost/reliability-aware TMVN	Selection + Simulator	Scenario comparison	R2/Final
Explainable human approval	Decision + Audit + Operator PWA	E2E approve/override trace	R2/Final
Reliable platform operations	AWS + CI/CD + Monitoring	Load, restore and alert evidence	Final

13. Final assumptions

AWS is the target cloud; responsive PWAs are the user channels; dispatches are simulated or operator-approved academic demonstrations; and external data is conditional on current rights and institutional approval. These constraints are design safeguards, not temporary omissions.